EXPERIMENT NO: 08

TITLE: Develop a Program to Create a Reverse TCP Connection Using Python Sockets

1. AIM

To design, implement, and verify a TCP socket-based reverse connection demonstration in Python using a listener and a connector running on localhost.

2. PREREQUISITES

• Basic understanding of client-server architecture and TCP communication.

• Knowledge of Python input/output operations and function calls.

• Basic understanding of IP addresses, ports, and sockets.

• Python installed and Command Prompt available on the system.

3. ALGORITHM / SYSTEM PROCEDURE

Listener Program Procedure (listener.py):

1. Import the socket module.
2. Set the host address to 127.0.0.1 and port to 5000.
3. Create a TCP socket using socket.AF_INET and socket.SOCK_STREAM.
4. Bind the socket to the localhost address and port.
5. Listen for one incoming TCP connection.
6. Accept the connection and display the connector address.
7. Receive a message from the connector and display it.
8. Send a reply message back to the connector.
9. Close the connection and server socket.

Connector Program Procedure (connector.py):

1. Import the socket module.
2. Set the host address to 127.0.0.1 and port to 5000.
3. Create a TCP socket.
4. Connect to the listener at the specified localhost address and port.
5. Send a harmless test message to the listener.
6. Receive the reply from the listener.
7. Display the reply and close the socket.

4. PROGRAM CODE

Listener Code: listener.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
```

```
print("Waiting for reverse connection...")
conn, addr = server.accept()

print("Connected from:", addr)

message = conn.recv(1024).decode()
print("Received:", message)

conn.send("Hello from listener".encode())

conn.close()
server.close()
```

Connector Code: connector.py

```
import socket

HOST = "127.0.0.1"
PORT = 5000

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

print("Connecting to listener...")

client.connect((HOST, PORT))

client.send("Hello from reverse connector".encode())

reply = client.recv(1024).decode()

print("Listener replied:", reply)

client.close()
```

5. EXECUTION STEPS & OUTPUT

Step 1: Start the Listener Process

Open Command Prompt in the experiment folder and run:

```
python listener.py
```

Listener waits for the reverse connection.

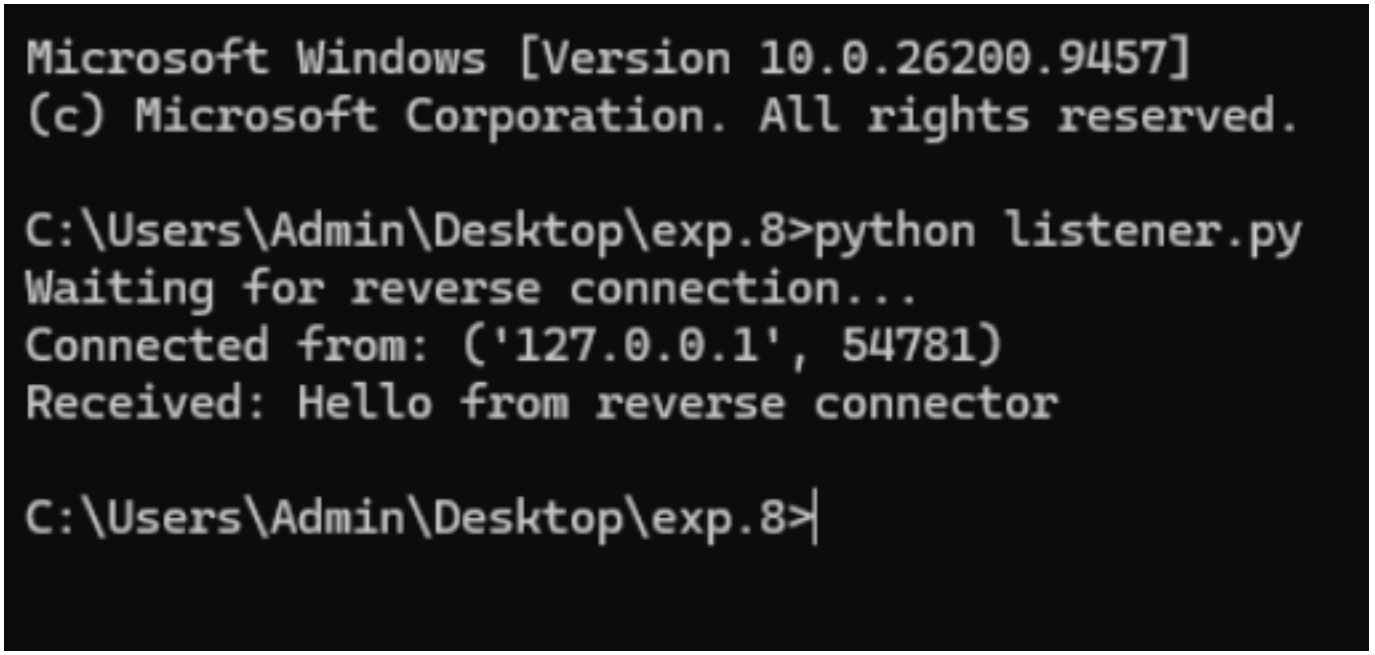


Figure 1: Actual listener terminal output after receiving the TCP connection.

Step 2: Start the Connector Process

Open a second Command Prompt in the same experiment folder and run:

```
python connector.py
```

The connector initiates the TCP connection and exchanges messages.

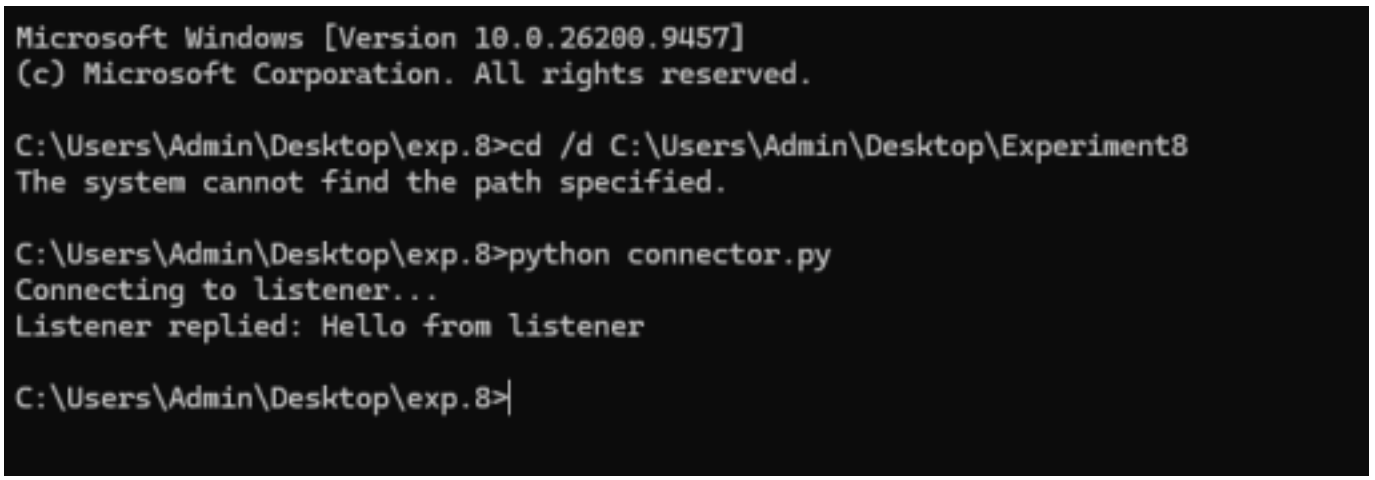


Figure 2: Actual connector terminal output showing the listener reply.

6. FINAL OUTPUT

Listener Terminal Output:

```
Waiting for reverse connection...
Connected from: ('127.0.0.1', 54781)
Received: Hello from reverse connector
```

Connector Terminal Output:

```
Connecting to listener...
Listener replied: Hello from listener
```

7. RESULT

The TCP socket-based reverse connection demonstration was successfully executed. The connector initiated a TCP connection to the listener on localhost (127.0.0.1:5000), and messages were successfully exchanged between the two programs.

Note: This experiment demonstrates the reverse TCP connection concept safely on localhost using message exchange. It does not implement arbitrary remote command execution.